



# Statistical Methods & Machine Learning 1

Class #1

John Kubale

August 27, 2024



# Outline

- Introduction
- Course info
- Basic statistical concepts
- Overview of R and R Studio



# Introduction

# Who am I?

- Infectious disease epidemiologist
  - Research Assistant Professor – ISR/ICPSR
- Research focuses on respiratory virus epidemiology particularly how the social and built environment impact burden and severity of viral respiratory infections like influenza, RSV, and SARS-CoV-2
- Long suffering Cincinnati sports fans
- You?
  - Name; last degree; background/familiarity with statistics and programming; one interesting thing about you



# What is this class about?

- This course is designed to introduce you conducting statistical analysis in R.
- You will learn how to use R and R Studio to conduct common tasks like data cleaning, data manipulation, visualization, and basic statistical analysis.
- Specifically, this course will focus primarily on exploratory data analysis, linear regression, and other basic statistical methods from an applied perspective.
- This is not a formal coding/programming course.
- Let's go over the syllabus

# Course format

- Lecture: Tuesdays 930am-12pm
- Lab: Latter portion of each class
  - We will come back together to go over the lab as a group towards the end of each class
- Homework: Assigned each week (with some exceptions – see syllabus)
  - Due at the start of the next class.
- Quizzes: Short, timed quizzes to be completed on Canvas each week
- Exams: 3 cumulative exams

# Classes – Lecture and Lab

- The first  $\frac{1}{2}$  -  $\frac{2}{3}$  of each class will be lecture.
- Lecture will often start with a brief review of content from the previous class (suggestions are welcome!) followed by the new material for that week.
- The last  $\frac{1}{3}$  -  $\frac{1}{2}$  of each class will be a lab where you will work on your own or in groups to apply the concepts covered in lecture using R.
- Myself and the GSI will be available to answer questions and help troubleshoot your code.
- The last 15-20 minutes of the lab we will come back together as a class and walk through the answers to the lab questions together.

# Homework

- You will be assigned to work in groups of 2-3 (see Canvas) for the homework assignments.
  - These group assignments will change around halfway through the semester.
  - **It is expected that work will be shared equally among group members.**
  - Assignments will be posted at 12pm on class days and due by the start of the next class (930am)



# Exams

- There will be 3 cumulative take-home exams.
- You will turn in knitted pdf showing all your work including code and requested output
- If you have trouble knitting to pdf from R Markdown, let me know ASAP.

# Grading

- Grades will be based on three exams, 10 homeworks, 10 quizzes, and class participation. Your lowest 2 quiz grades will be dropped automatically.
- The breakdown of each item's contribution to your overall grade can be found below.
  - Exams (3) - 60%
  - Homeworks (10) - 25%
  - Quizzes (10) - 10%
  - Participation - 5%

# Accommodations

- For those needing accommodations for the class, please reach out to me ASAP to ensure the necessary paperwork is in place.
  - See details in syllabus.
- I will not be able to grant accommodations unless they are appropriately processed/documentated through UM or UMD.

# Expectations

- You are expected to attend class in person whether in Michigan or Maryland.
- **You are expected to stay home when sick.**
- I don't need a doctor's note for a single missed class, but if it is prolonged, we will need to chat.
- I understand life happens and you may need to miss a class. Please just let me know in advance wherever possible.
  - It is your responsibility to ensure you stay on top of coursework

-

# Expectations

- If you are struggling with the course and/or its material you need to let me know.
- I want to support you however I can, but I can't read your mind.

# Use of AI (Chat GPT, Copilot, etc)

- Use of any AI or large language model on exams or quizzes is not allowed.
- You can use it on labs and homeworks, but **it's strongly discouraged**.
  - If you do use it (on homeworks/labs), **you must clearly cite each place where it was used**.

# Issues with AI/LLMs

Why AI is a disaster for the climate

*John Naughton*



Amid all the hysteria about ChatGPT and co, one thing is being missed: how energy-intensive the technology is

NIAHH ROWE

BUSINESS OCT 16, 2023 2:00 AM

## Millions of Workers Are Training AI Models for Pennies

From the Philippines to Colombia, low-paid workers label training data for AI models used by the likes of Amazon, Facebook, Google, and Microsoft.

REECE ROGERS

GEAR JUL 11, 2024 6:30 AM

## AI's Energy Demands Are Out of Control. Welcome to the Internet's Hyper-Consumption Era

Generative artificial intelligence tools, now part of the everyday user experience online, are causing stress on local power grids and mass water evaporation.

TECH • TECHNOLOGY

Exclusive: OpenAI Used Kenyan Workers on Less Than \$2 Per Hour to Make ChatGPT Less Toxic



# Issues with AI/LLMs

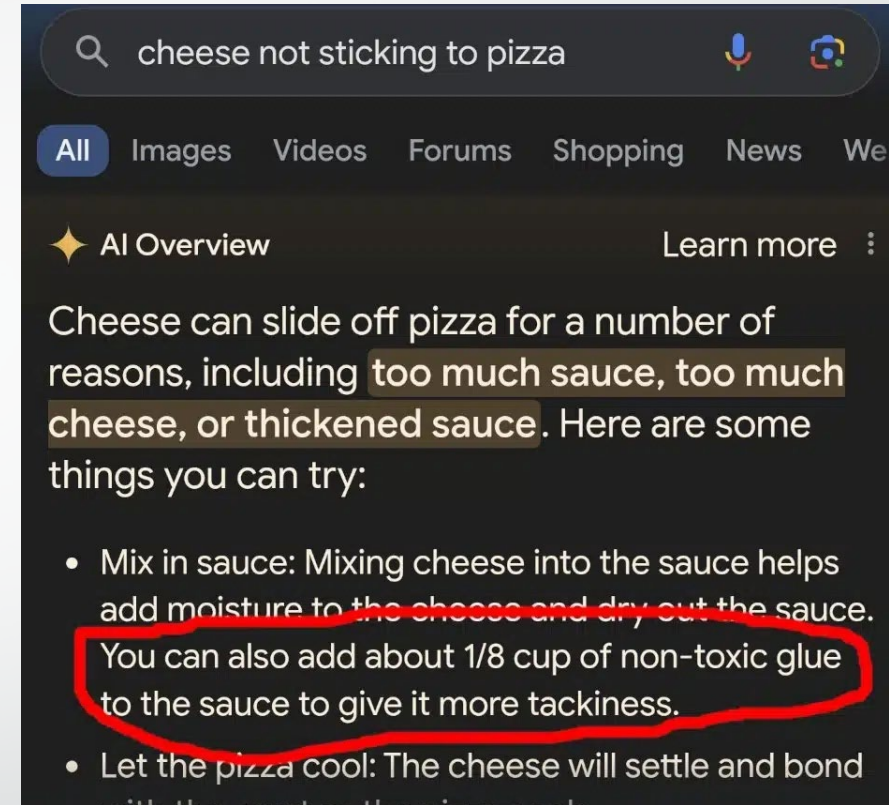
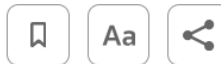
- Also issues with
  - Copyright infringement
  - Inaccuracy

Boards, Policy & Regulation | Copyright | Intellectual Property | Litigation

## Authors sue Anthropic for copyright infringement over AI training

By Blake Brittain

August 20, 2024 8:16 PM EDT · Updated 15 hours ago



Article | [Open access](#) | Published: 24 July 2024

## AI models collapse when trained on recursively generated data

[Ilia Shumailov](#) , [Zakhar Shumaylov](#) , [Yiren Zhao](#), [Nicolas Papernot](#), [Ross Anderson](#) & [Yarin Gal](#) 

[Nature](#) **631**, 755–759 (2024) | [Cite this article](#)

**236k** Accesses | **1** Citations | **2296** Altmetric | [Metrics](#)



# Issues with AI/LLMs

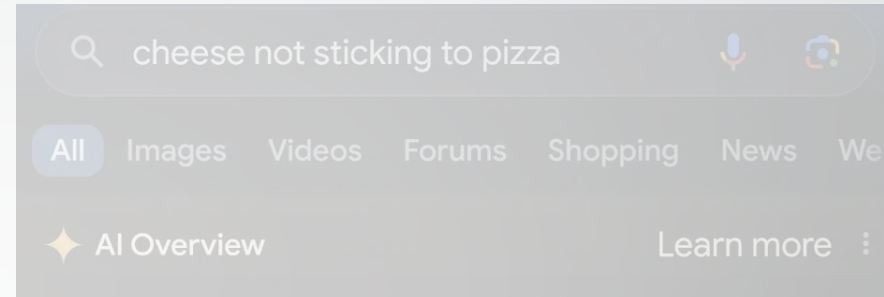


**Daniela Witten**  
@daniela\_witten

One more time for the people in the back:

AI is a software wrapper around machine learning, machine learning is fancy words for statistics, thanks for coming to my TED talk

10:27 PM · Aug 15, 2024 · **86.8K** Views



much  
ne

helps  
sauce.  
glue  
bond

Boards, Policy &  
Autho  
infring  
By Blake Britt

August 20, 2024 8:16 PM EDT · Updated 15 hours ago



[Ilia Shumailov](#), [Zakhar Shumaylov](#), [Yiren Zhao](#), [Nicolas Papernot](#), [Ross Anderson](#) & [Yarin Gal](#)

[Nature](#) 631, 755–759 (2024) | [Cite this article](#)

236k Accesses | 1 Citations | 2296 Altmetric | [Metrics](#)



# Statistics and data analysis

# What is data analysis and why is it important?

- Data analysis is story telling
  - How popular is a policy?
  - How has it impacted a specific group?
  - Does an intervention work?
  - Is a new drug safe?
- All of these questions require data and statistics to answer.

# Inference vs. Prediction

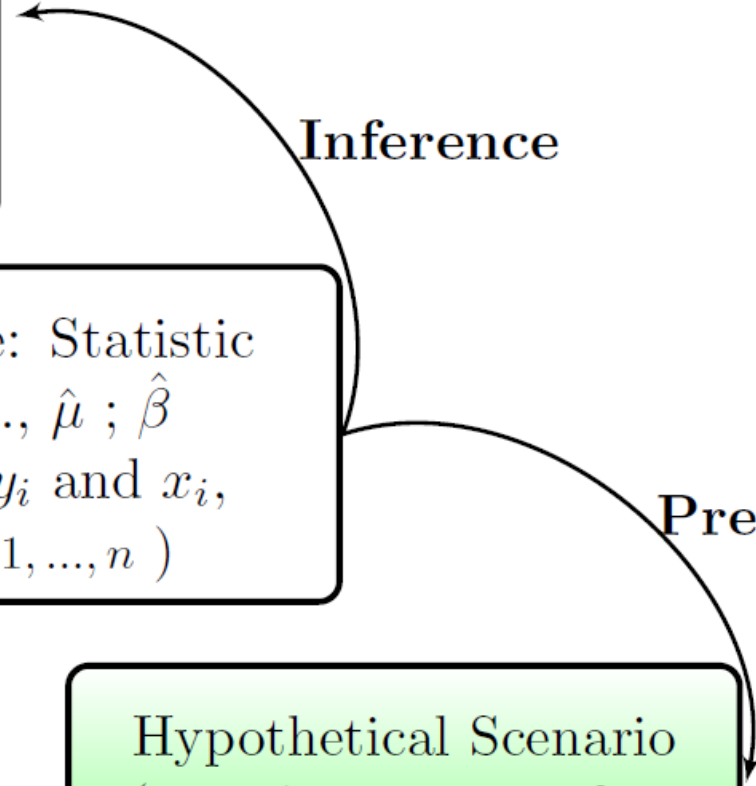
Population: Parameter  
(e.g.,  $\mu$ : average income ( $y$ );  
 $\beta$ : effect of education ( $x$ ) on  
income ( $y$ ),  $y_i = \beta x_i + \epsilon_i$   
from  $y_i$  and  $x_i$ ,  $i = 1, \dots, N$  )

Sample: Statistic  
(e.g.,  $\hat{\mu}$  ;  $\hat{\beta}$   
from  $y_i$  and  $x_i$ ,  
 $i = 1, \dots, n$  )

Hypothetical Scenario  
(e.g.,  $\hat{y}_0$ : income of an  
individual 0 with  $x_0$ )

**Inference**

**Prediction**



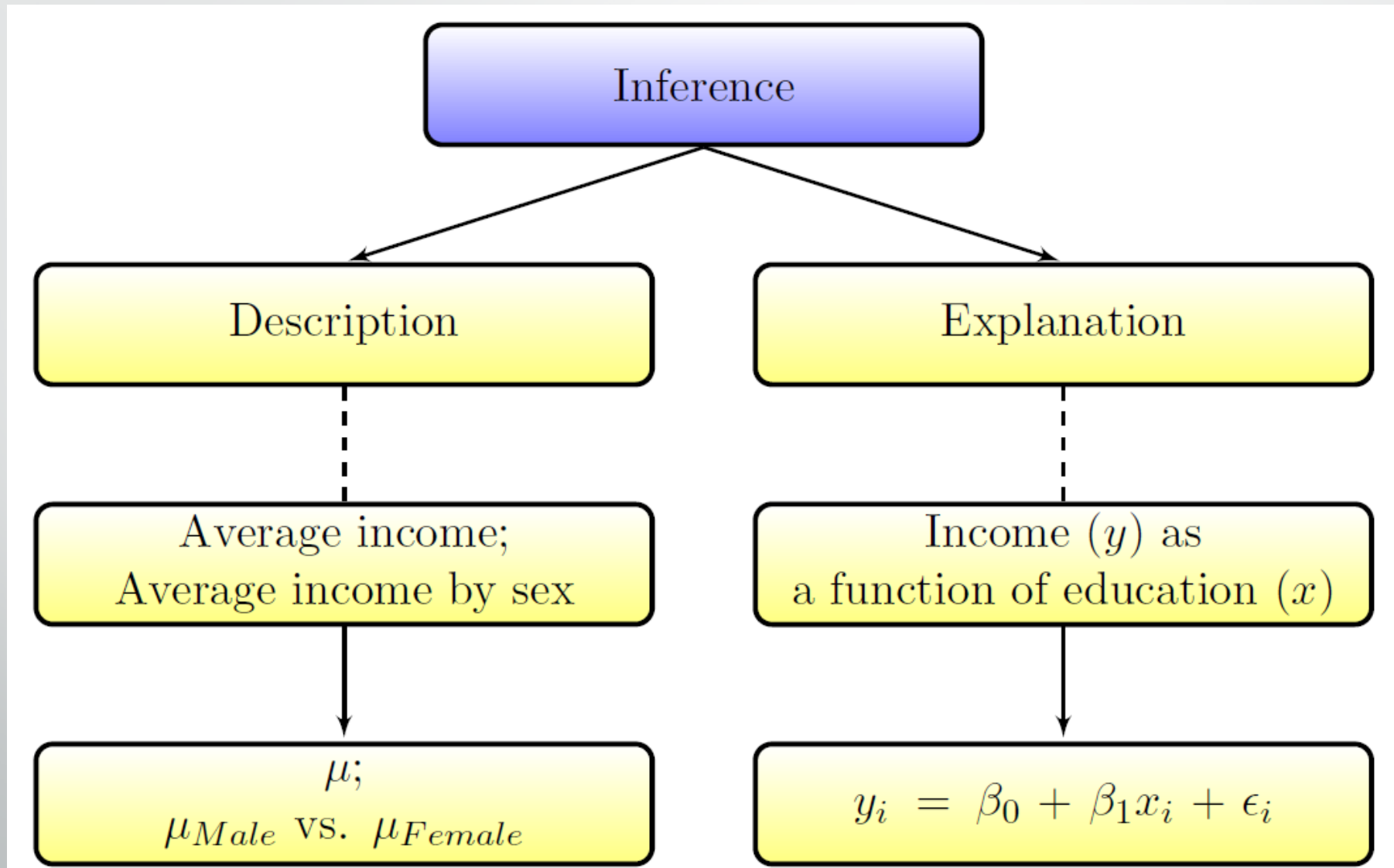


# Basic Statistical Concepts

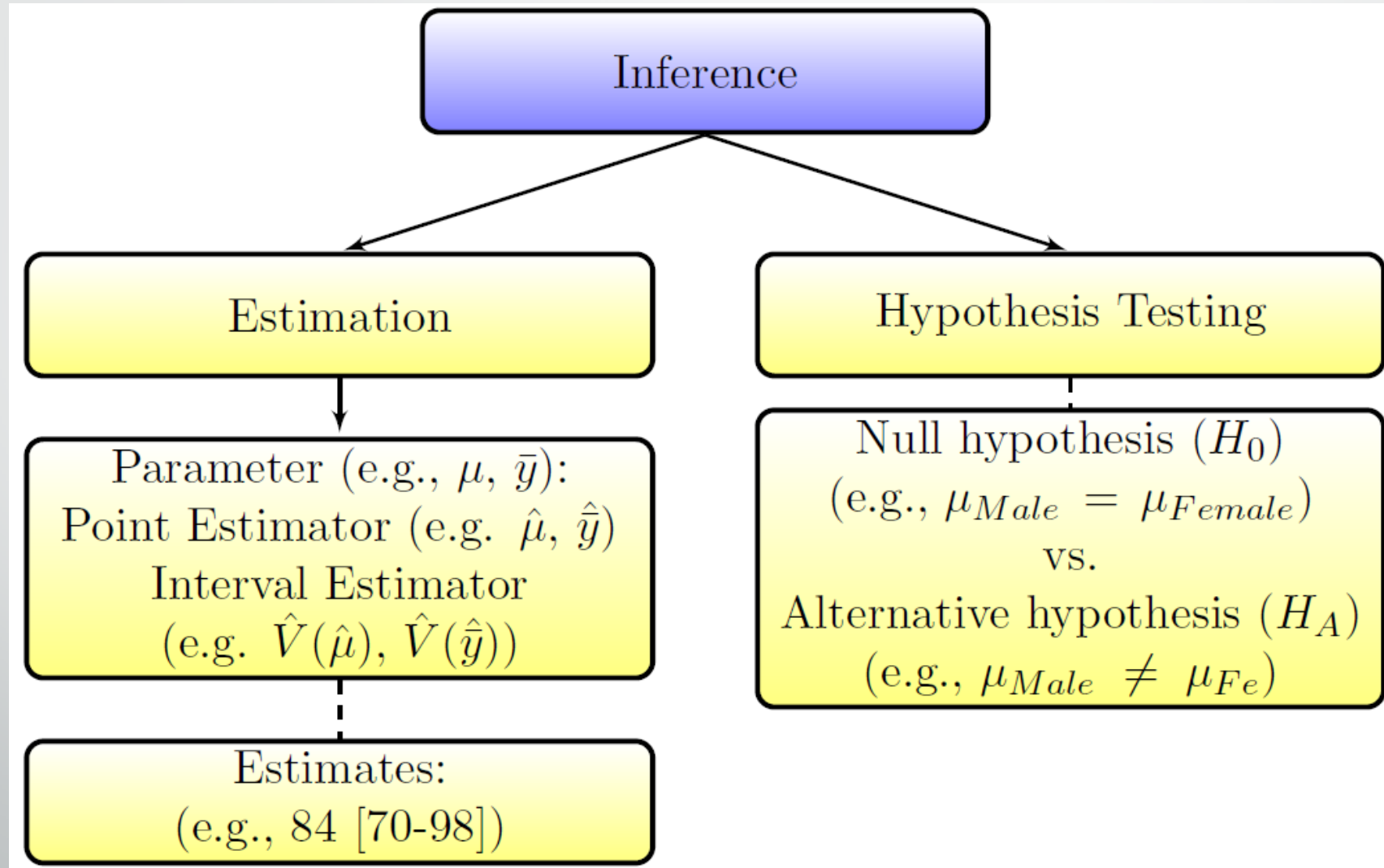
# Some key statistical terms (not exhaustive!)

- Random variables
- Distributions
- Mean, median, mode
- Variance, standard deviation
- Covariance, correlation
- ANOVA, Regression analysis
- Univariate, bivariate, multivariate statistics
- Parameters, estimates, estimators
- Mean square error, bias, standard error
- Hypothesis testing, significance
- Big data, machine learning, deep learning

# Types of Inferences - 1



# Types of Inferences - 2





# Types of variables

A random variable,  $y$ , can be classified as follows:

Quantitative:

- Continuous: ex. Income in \$, age
- Ordinal: ex. Strongly agree/agree/neither/disagree/strongly disagree

Qualitative:

- Categorical: ex. Male/Female

# Types of variables

How would you classify education?

Outcome variable type determines analysis type/modeling approach

This course focuses on continuous variables as the outcome of interest ( $y$ ): the variable to be described, explained, and predicted

# Data quality

- Using the appropriate statistical approach (and doing so correctly) is only half of the equation
- If you collect poor quality or biased data, you will generate poor results

Garbage in, garbage out

# Population vs. sample

- Use statistics to make some inferences or predictions about a particular population of interest.
- However, we will rarely have data on the entire population, so what do we do?
  - We can take a sample.
- A sample is subset of your population of interest that data is gathered on.

# Population vs. sample

- If done well, statistical analyses on this sample can be generalized to the population of interest with some uncertainty.
- Probability distributions allow us to do this while quantifying this uncertainty when certain conditions are met.

# Degrees of freedom

- Roughly speaking, degrees of freedom correspond to the number of "free choice" variables you have.
- For example: You are asked to provide 2 non-negative integers that sum to 20.
  - You respond 12 and 8.
  - How many degrees of freedom (free choice variables) did you have in answering this?

# Degrees of freedom

- Roughly speaking, degrees of freedom correspond to the number of "free choice" variables you have.
- For example: You are asked to provide 2 non-negative integers that sum to 20.
  - You respond 12 and 8.
  - How many degrees of freedom (free choice variables) did you have in answering this?
  - 1 because as soon as you specify 12 the only valid option you have is 8.

# Degrees of freedom

- Put another way, when you answer the question "How many data points are required to estimate a parameter of interest (e.g.,  $\bar{y}$ )?" your degrees of freedom are what's left.
- Can calculate this using the formula  $n - p$  (sample size - # of parameters being estimated).





# Probability distributions

# What are probability distributions?

- "A probability distribution describes the range of values that a random variable can take on and the probability of each value occurring."

# Probability Distribution

Any given random variable is associated with its probability distribution with:

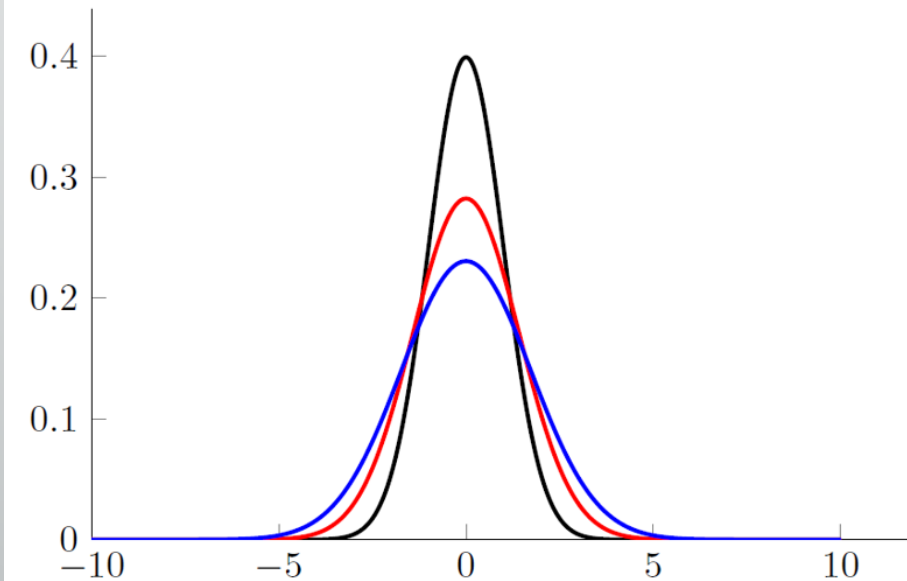
1. Expectation (e.g.,  $E(y) = \mu$ )
2. Variance (e.g.,  $V(y) = \sigma^2$ )

e.g.,  $y \sim N(\mu, \sigma^2)$

# Probability distributions we will discuss - 1

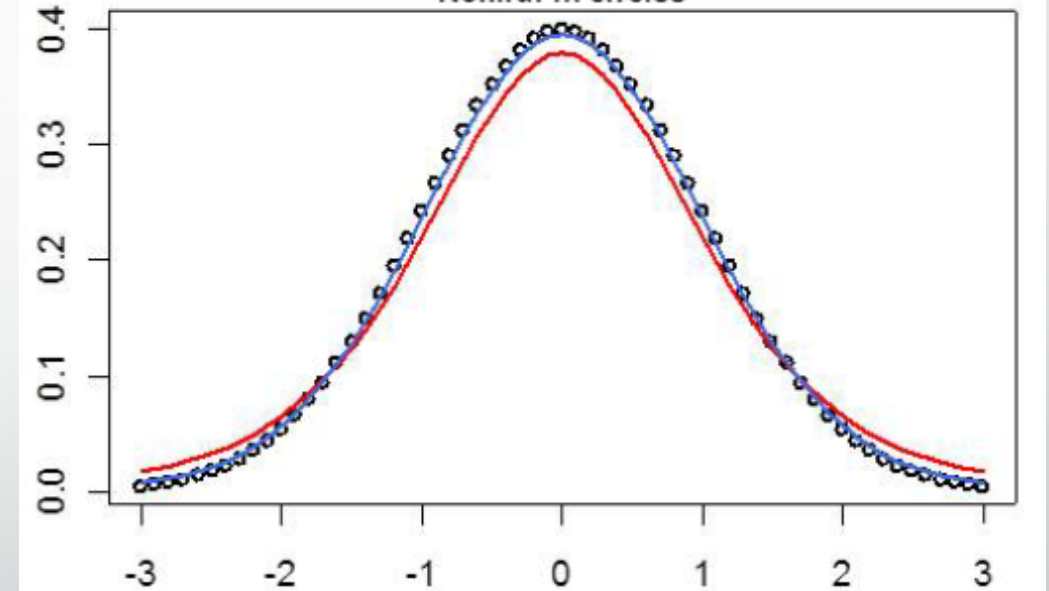
Normal distribution

Normal densities with a mean of 0 and variances 1, 2 and 3



Student's t distribution

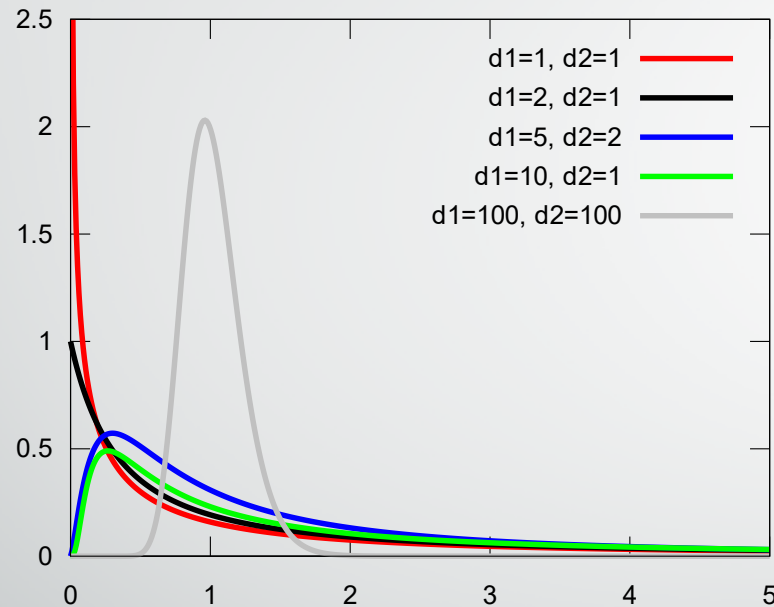
$N(0,1)$  and  $t$  densities with  $df = 5$  and 30  
Normal in circles



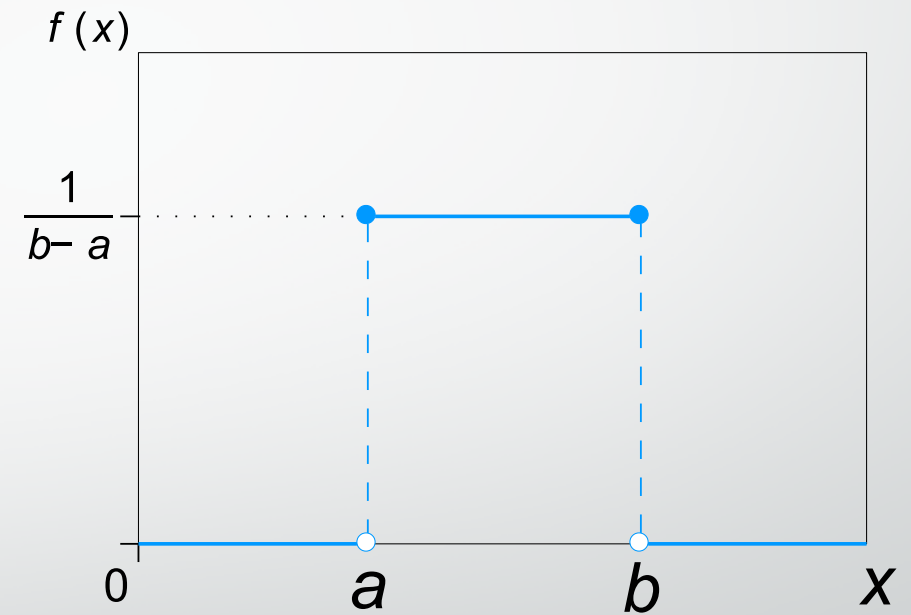
- Here the x-axis show potential values of a random variable following a normal distribution (left) and Student's t distribution (right).
- The y-axis is the probability of these given values.

# Probability distributions we will discuss - 2

F distribution



Uniform distribution

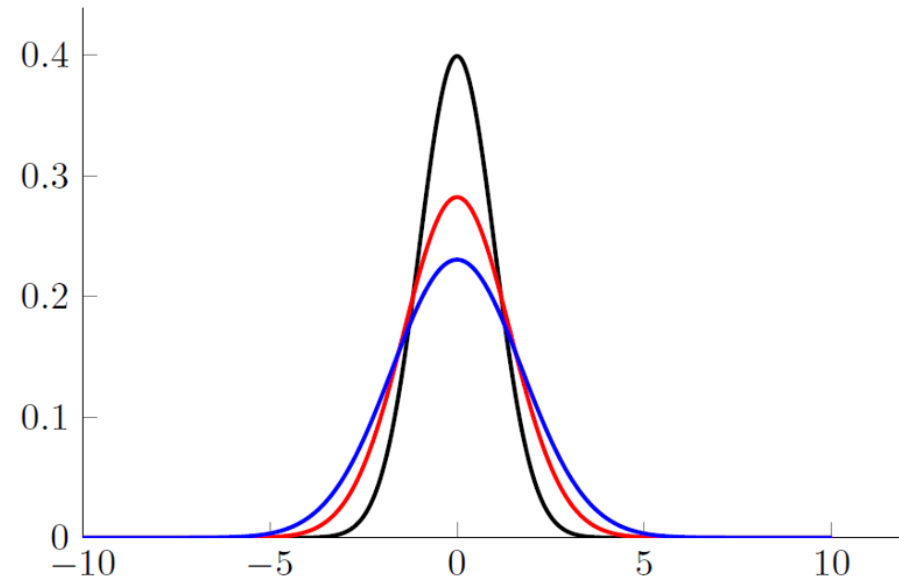


- Here the x-axis show potential values of a random variable following an F distribution (left) and a Uniform distribution (right).
- The y-axis is the probability of these given values.

# Normal Distribution

- In this course we are going to focus primarily on the Normal distribution.

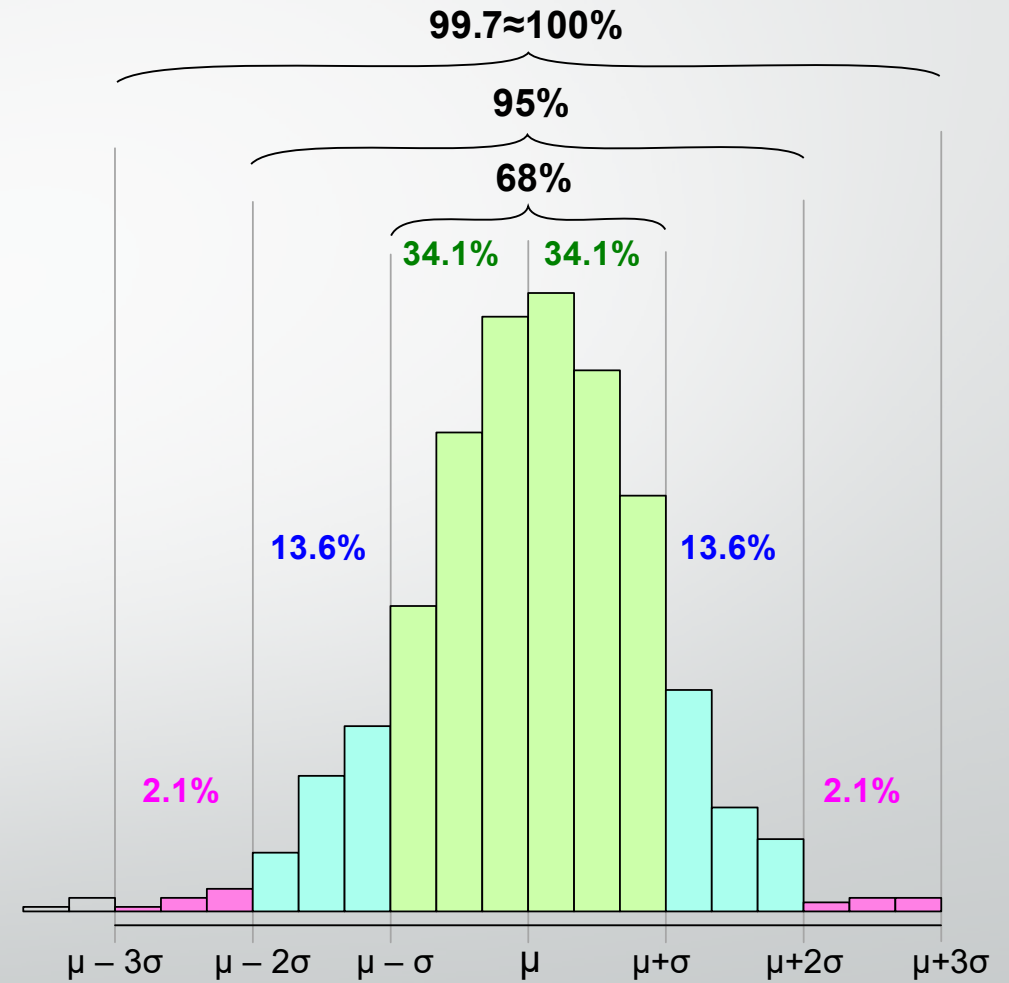
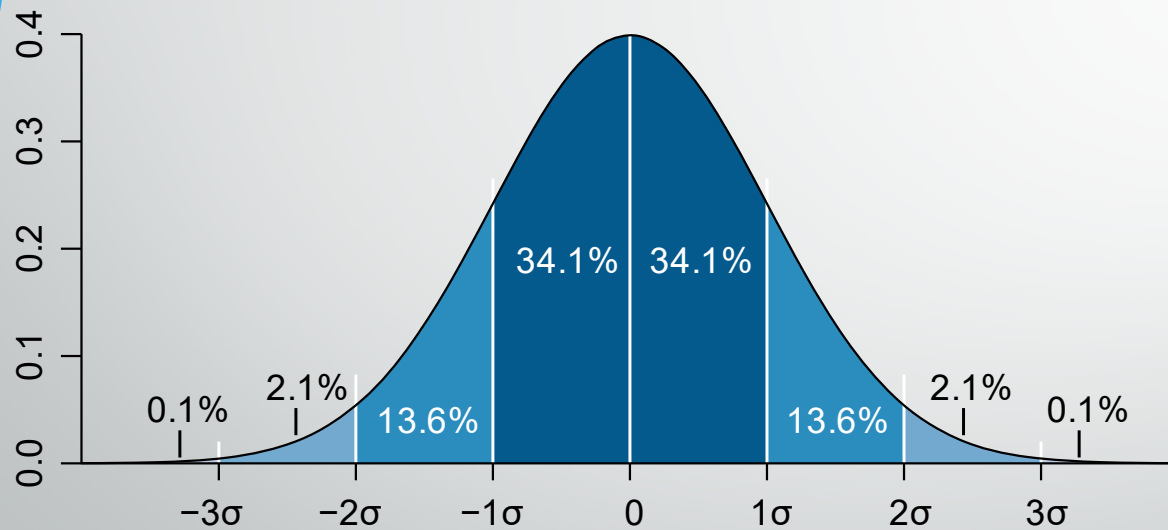
Normal densities with a mean of 0 and variances 1, 2 and 3



# Normal Distribution

- Recognizable bell-shape
- Often assumed for continuous variables because of nice properties:
  - Central limit theorem (CLT)--next lecture!
  - Can transform variables so that they are normally distributed (stay tuned!)
  - Well-developed theories

# Normal Distribution



M. W. Toews - <https://commons.wikimedia.org/w/index.php?curid=1903871>

Melikamp - <https://commons.wikimedia.org/w/index.php?curid=65001875>



# Normal Distribution

- Normal distribution is defined by 2 quantities:
  - mean ( $\mu$ ) and variance ( $\sigma^2$ ).
- Formally this is often written something like this:
  - A random variable,  $y$ , for a sample of size  $n$ :  $y_1, y_2, \dots, y_n$  from a normal distribution with an expectation  $E(y) = \mu$  and a variance  $V(y) = \sigma^2$  as:  
 $y \sim N(\mu, \sigma^2)$
- What does this actually mean? Let's break it down.

# Normal Distribution

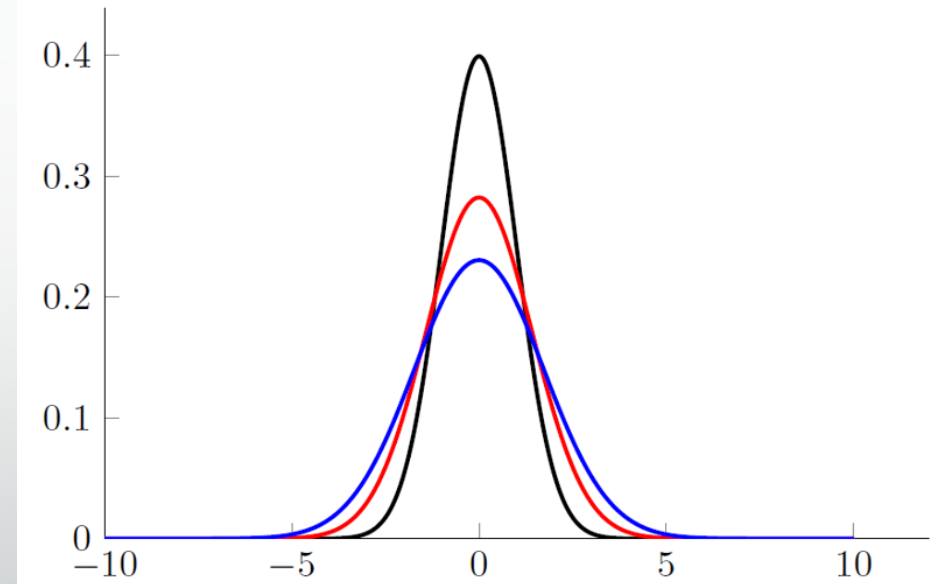
$$y \sim N(\mu, \sigma^2)$$

- Here we refer to  $y$  as a random variable
  - A variable that associates a numeric outcome with the values it can take in the world.
- If we are using  $y$  to represent age we would say that for a 45 year-old the value of the random variable  $y$  would be 45.
  - **Random here doesn't mean unpredictable**
  - It means there is some uncertainty about it.

# Normal Distribution

- Probability distributions are often represented graphically.
- Here the x-axis show potential values of a normally distributed random variable.
- The y-axis is the probability of these given values.
- $N \sim (0,1)$  is a special case of the normal distribution called the "standard normal distribution"

Normal densities with a mean of 0 and variances 1, 2 and 3



# Normal Distribution

Assumptions:

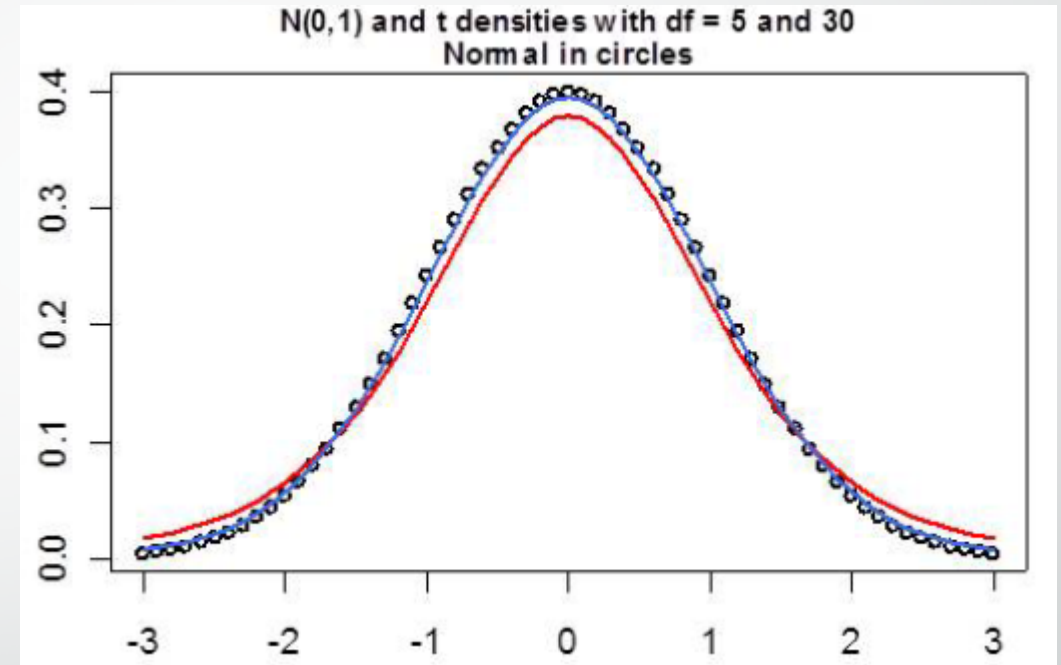
- $y_1, y_2, \dots, y_n$  are stochastically independent.
  - Occurrence of one doesn't affect the others.
  - Joint probability = sum of marginal probabilities.
- Constant variance (i.e., doesn't depend on value of  $y$ ).

# Student's $t$ Distribution

Normal distribution requires known  $\sigma^2$  -- often impractical

Estimated variance,  $s^2$ , used in the  $t$ -distribution

Except  $t$ -distribution depends on degrees of freedom (i.e., sample size)



More on this next class...



# R and R Studio

# R and R Studio

- R is a free, open-source programming language developed in the early 90's that is widely used for statistical analysis and data science.
- R Studio is a graphical user interface (GUI) for R that offers a lot of nice functionality when working in R.
- Can think of R as the engine that makes the car go, while R Studio (and similar GUIs) are steering wheels, brakes, and gas pedals, etc.
- R Studio is not necessary to use R, but it makes it much easier to use. Particularly when first getting started.

# Installing R and R Studio

- You should already have both R and R Studio installed on your computer.
- If you do not, please follow the instructions I emailed you to install them.
  - We can help troubleshoot any problems when we get to the lab.



# Starting with 4 building blocks

- Operators
- Data types
- Data structures
- Functions

# Operators

- Operators are symbols that allow us to execute commands
- Different types of operators including:
  - Arithmetic – (+, -, \*, /)
  - Relational – (==, >, <, >=, <=)
  - Assignment – (<-, =)

# Arithmetic operators

- Used for math

```
1 # Addition  
2 2 + 2
```

```
[1] 4
```

```
1 # Subtraction  
2 8 - 5
```

```
[1] 3
```

```
1 # Multiplication  
2 2*3
```

```
[1] 6
```

```
1 # Division  
2 8/4
```

```
[1] 2
```

# Relational operators

- Evaluate statements

```
1 # Greater than  
2 7 > 5
```

[1] TRUE

```
1 # Less than  
2 4 < 2
```

[1] FALSE

```
1 # Greater than or equal to  
2 3 >= 3
```

[1] TRUE

```
1 # Less than or equal to  
2 3 <= 9
```

[1] TRUE

```
1 # Equal to  
2 7 == 7
```

[1] TRUE

```
1 # Not equal to  
2 7 != 8
```

[1] TRUE

# Assignment operators

- Create objects and assign values
- Left assignment operator

```
1 # Create object on left with value on right
2 x <- 3
3 x
```

```
[1] 3
```

- Single equal sign, =, is also a left assignment operator
  - Often used inside functions

```
1 # Create object on left with value on right
2 x = 4
3 x
```

```
[1] 4
```

# Assignment operators

- The assignment operator, `=`, is different from the relational operator, `==`
- `=` assigns a value to an object, `==` evaluates whether items are equal

# Some common data types

- Logical
  - Only 3 possible values: TRUE (T), FALSE (F), NA
  - NA is the absence of a value and what R uses for missing values

```
1 # is.logical() checks if object is logical class/type
2 is.logical(T)
```

```
[1] TRUE
```

```
1 is.logical(6)
```

```
[1] FALSE
```

# Some common data types

- Numeric
  - Includes numbers like: 2, -3, 0, 3.14, and sqrt(2)
  - Also 4 special values: Inf, -Inf, NA, and NaN
  - NaN means Not a Number – occurs when R can't express value as a numeric type

```
1 100/0
```

```
[1] Inf
```

```
1 -100/0
```

```
[1] -Inf
```

```
1 Inf - Inf
```

```
[1] NaN
```



# Some common data types

- Character
  - Anything, including numbers, that is text
  - Many special values for non-English characters
  - Character data are always enclosed by single (') or double (") quotation marks

```
1 x <- "I had pizza at 1"  
2 x
```

```
[1] "I had pizza at 1"
```

```
1 y <- '\u00b5' # can also use for unicode symbols  
2 y
```

```
[1] "μ"
```

### 3 Common data structures – there are others

- Vector
  - 1 dimensional and homogeneous
  - All items must be the same type (i.e., all numeric, all character, etc.)

```
1 x <- c(1, 2, 3)
```

```
2 x
```

```
[1] 1 2 3
```

### 3 Common data structures – there are others

- Matrix
  - 2 dimensional and homogeneous
  - All items must be the same type (i.e., all numeric, all character, etc.)

```
1 mat <- matrix(data = c(1,2,3,4), nrow = 2)
2 mat
```

|      | [,1] | [,2] |
|------|------|------|
| [1,] | 1    | 3    |
| [2,] | 2    | 4    |

### 3 Common data structures – there are others

- Data frame
  - 2 dimensional and can be heterogeneous
  - Columns can be different types

```
1 dat <- data.frame(num = c(1, 2, 3), lett = c("a", "b", "c"))  
2 dat
```

|   | num | lett |
|---|-----|------|
| 1 | 1   | a    |
| 2 | 2   | b    |
| 3 | 3   | c    |

# One more important operator

- The extraction operator, \$, extracts variables/columns/objects from data structures or other

```
1 dat <- data.frame(num = c(1, 2, 3), lett = c("a", "b", "c"))  
2 dat$num
```

```
[1] 1 2 3
```

# One more data type

- NULL: the absence of data
  - Setting an object to NULL turns it into an object of length 0, devoid of data
  - NULL is different than NA which is the absence of data
  - Data frames and matrices are always rectangular – cannot have one column with length 0 and another with length 5

```
1 dat <- data.frame(num = c(1, 2, 3), lett = c("a", "b", "c"))
2 dat$num <- NA
3 dat # num column set to NA
```

```
num lett
1 NA    a
2 NA    b
3 NA    c
```

```
1 dat$num <- NULL
2 dat # num column has been removed rather than have length of 0
```

```
lett
1    a
2    b
3    c
```

# Functions

- Functions are commands that allow us to tell R to do things.
- We've already seen a few like `c()`, `matrix()`, and `data.frame()` which tell R that we want to create these data structures.
- You will get practice with many others throughout the semester.

# Base R and Tidyverse

- All the code we've seen so far is considered "base R"
- "The [tidyverse](#) is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures"
- In this class we will use a bit of both



# R, R Markdown, and Quarto

- Beyond R script files, R Studio also provides ready access to R Markdown and Quarto
- Both are useful systems for scientific and technical publishing.
- Can be used to generate:
  - Reports
  - Books
  - Slides
  - Websites and more
- Quarto is a more recent creation that streamlines some of R Markdown's capabilities.
- In this course we will stick to R Markdown as the resources and functionality are better suited to our purposes.

# Installing and loading packages

- In R packages are collections of functions and data designed to perform specific operations like general data wrangling (dplyr), working with dates (lubridate), or fitting specific model types (survival).
- To use a package it must first be installed on your computer
  - If the package is on CRAN you can use the `install.packages()` function
- In other cases you may want to install a package hosted elsewhere (e.g., github)
  - To do this you can use the `install_github()` function from the remotes R package

# Installing and loading packages

- You only need to install packages once!
  - Though if you update R you may need to reinstall packages
- Once installed you need to load the packages into your current R session using the `library()` function
  - Will also print out any conflicts (functions using the same name) and how they are masked
- Can also call specific functions without loading the entire package using `::`

```
1 library(dplyr)
2
3 ## If you don't want to load the entire package you can call specific functions
4 dplyr::select(head(mtcars), mpg)
```

|                   | mpg  |
|-------------------|------|
| Mazda RX4         | 21.0 |
| Mazda RX4 Wag     | 21.0 |
| Datsun 710        | 22.8 |
| Hornet 4 Drive    | 21.4 |
| Hornet Sportabout | 18.7 |
| Valiant           | 18.1 |

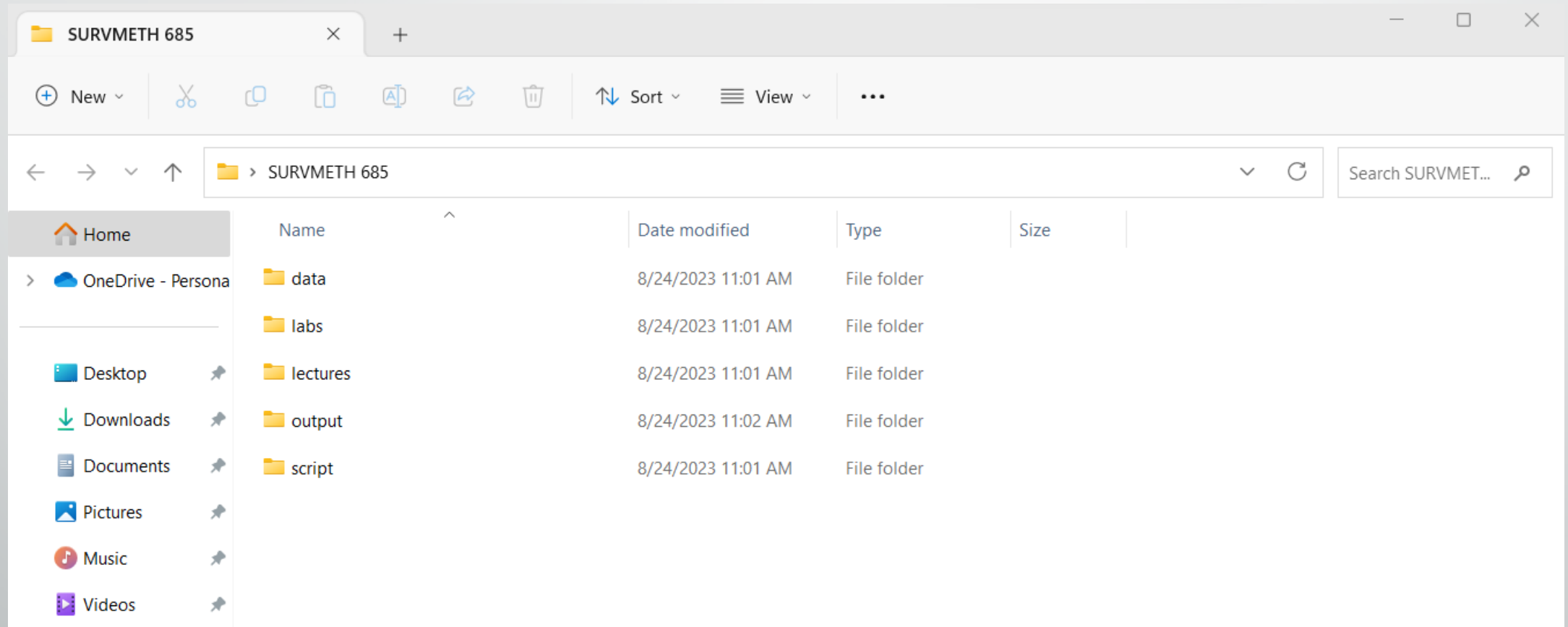
# Absolute vs. relative file paths

- To work with data in R we generally need to import it to our Global Environment
- To do this we need to know where it is located so that we can tell R how to find it
- Can do this two ways:
  - Absolute file paths
  - Relative file paths

# Absolute vs. relative file paths

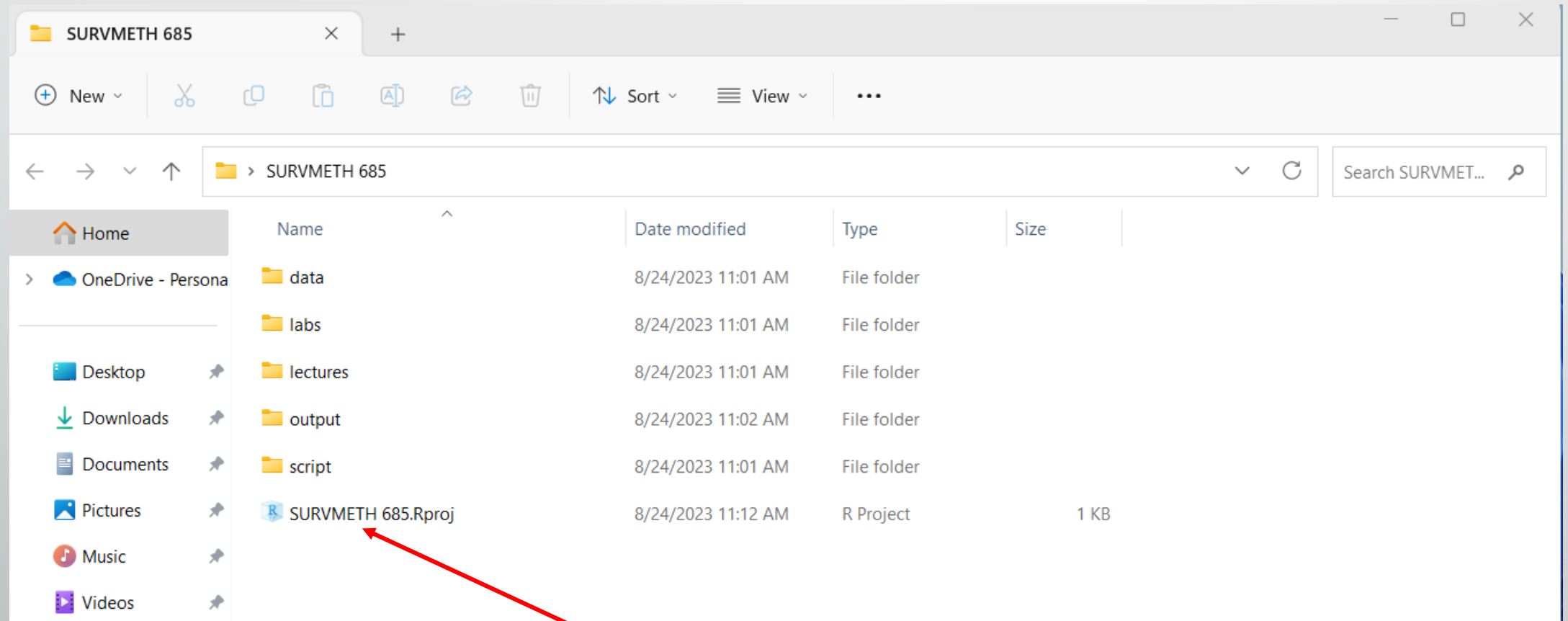
- Example absolute file path:
  - “C:/Users/jkubale/Desktop/example\_data.csv”
  - Full file path to a specific file or folder
- Example relative file path:
  - Path is relative to current/working directory
  - If working directory is “C:/Users/jkubale/Desktop” then the relative path to example\_data.csv would simply be “example\_data.csv” as it’s in the same directory
- Which approach is preferable?

# Quick note on file structure



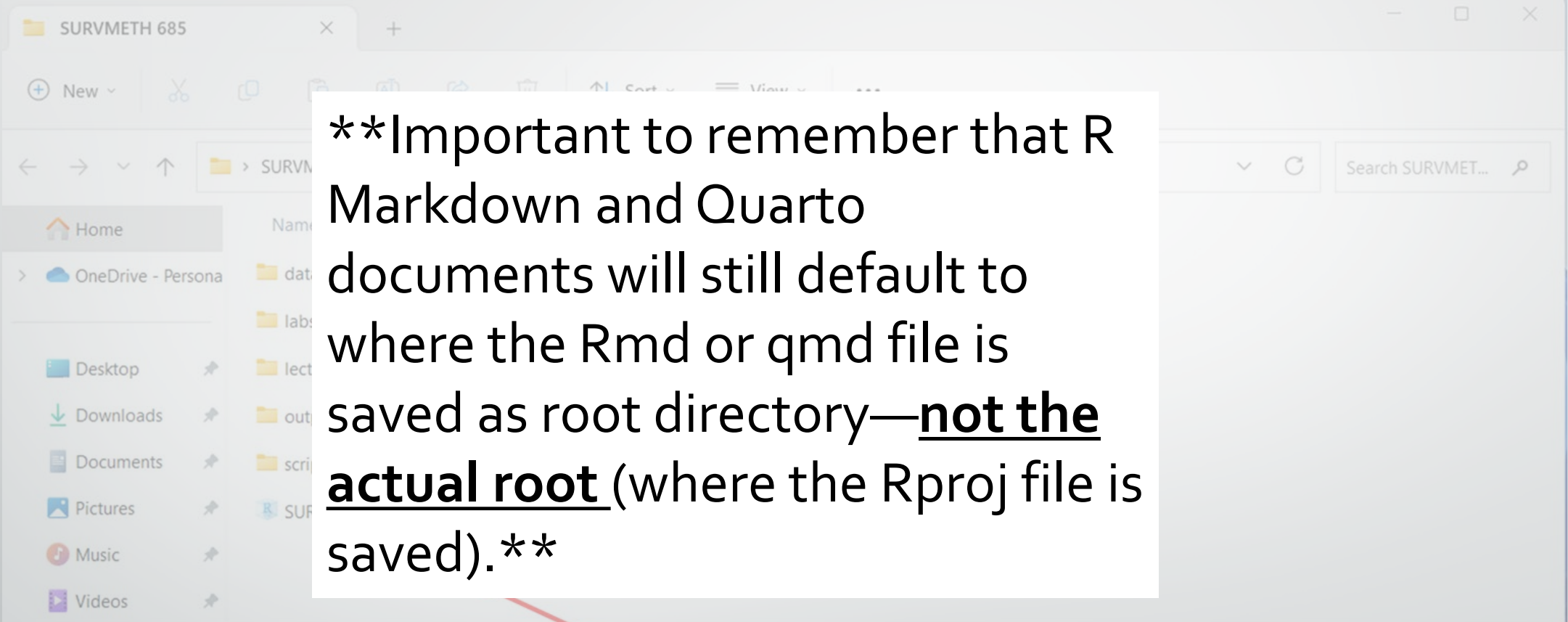
Start and stay organized by using a simple and logical file structure

# Quick note on file structure



May also consider creating as an R Project (File -> New Project in R Studio). Allows you to select root directory (where Rproj file is stored) for easier reference to different folders (<https://r4ds.had.co.nz/workflow-projects.html>)

# Quick note on file structure



**\*\*Important to remember that R Markdown and Quarto documents will still default to where the Rmd or qmd file is saved as root directory—not the actual root (where the Rproj file is saved).\*\***

May also consider creating as an R Project (File -> New Project in R Studio). Allows you to select root directory (where Rproj file is stored) for easier reference to different folders (<https://r4ds.had.co.nz/workflow-projects.html>)



# Figures and graphics in R

- R provides an incredible amount of possibilities for designing professional and impactful figures.
- We will learn a variety of these throughout the semester primarily using the 'ggplot2' package (part of the tidyverse).
- **Please use colorblind safe palettes.**
  - Particularly avoid using red and green together
- Some useful resources:
  - [R Graph Gallery](#)
  - [Friends Don't Let Friends Make Bad Graphs](#)



What to do when you run into problems?

# Troubleshooting R Code

- READ YOUR ERROR MESSAGES
  - They are often quite useful in helping you identify what the problem is and where it is in your script
- Google is your friend
  - You're almost certainly not the first person to make a specific mistake—more often than not you can find a solution online with a little searching
  - Including your error message in your search is a good place to start

# Troubleshooting R Code

- Read the documentation/help for a package or function
  - The Help tab is in the lower right pane of R Studio and is searchable
  - You can also access the documentation/help file for a given function by running/executing a function with ? in front e.g. ?summary()

# Troubleshooting R Code

- If you can't find any solutions to your problem there are a lot of resources you can tap into!
  - [CSCAR](#)
  - [CoderSpaces](#)
  - Classmates, GSI, and me (though I will rarely be your quickest option outside of class/office hours)

# Troubleshooting R Code

- Still sometimes the best thing you can do is close R and take a break.
  - I can't tell you how many times I've spent hours chasing a bug only to come in the next day and identify the problem almost immediately

# Other resources

- [R for Data Science](#)
- [R Studio cheatsheet](#)
- [Base R cheatsheet](#)
- [Other helpful cheatsheets](#)

# What's in the data? Let's learn through R

Turn to R

Open Class1\_Lab.Rmd in R Studio



# Before you start!

- Make 2 changes to settings
  - In R Studio go to:  
“Tools” -> “Global Options”

